

CONTENTS

Práctico 9 (E/S programada e Interrupciones) — Arquitectura de Computadoras	
FING UdelaR · Resolución completa y verificada	
Preguntas teóricas	
Ejercicio 1 — Polling del teclado	
Ejercicio 2 — Bomba de agua (activar / desactivar)	
Ejercicio 3 — Rutinas por secuencia de teclado (<ESC><n>)	
Ejercicio 4 — Tres dispositivos, una sola línea de interrupción (atención por nivel)	
A) Prioridad fija: $\text{pri}(\text{disp1}) > \text{pri}(\text{disp2}) > \text{pri}(\text{disp3})$	
B) Prioridad variable: existe $\text{pri}(\text{dispositivo})$	
Ejercicio 5 — Concentrador de comunicaciones (4 entradas, 1 salida)	
Ejercicio 6 — Iluminación de pasillos (botón + timer)	
Ejercicio 7 — Sistema de seguridad de gas (sensor + extractor + válvula)	
Ejercicio 8 — Caja de seguridad con combinación	
Resumen de técnicas usadas en este práctico	

Práctico 9 (E/S programada e Interrupciones) — Arquitectura de Computadoras

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ **Nota general sobre el material oficial:** el PDF “Ejercicios Resueltos” de este práctico tiene **errores reales** que ya se habían detectado al armar la guía temática de E/S (M7) — el Ejercicio 1 compara `== 1` contra un AND con `0x80` (imposible, la máscara solo puede dar `0x00` u `0x80`) y el Ejercicio 2 tiene comparaciones sin paréntesis alrededor del `&` que caen víctimas de la precedencia de operadores en C (`!= / ==` se evalúan antes que `&`). Al revisar **todo** el práctico para esta resolución (no solo esos dos puntos) aparecieron **errores adicionales**: un bug de asignación vs. comparación (`=` en vez de `==`) en el Ejercicio 6, constantes indefinidas por tipos en el Ejercicio 7, macros con `;` colgante en el Ejercicio 3, y una variable no inicializada usada como flag booleano más un desorden LIFO/FIFO en el buffer del Ejercicio 5. Todos se detallan en su ejercicio correspondiente, con la corrección y su verificación explícita. Todas las máscaras de este documento se comprobaron con `python -c "print(hex(...))"` antes de escribirse.

Preguntas teóricas

(a) Valores en los buses durante una operación de E/S — E/S aislada vs. mapeada a memoria.

Bus	E/S Aislada (<code>in</code> / <code>out</code>)	E/S Mapeada a Memoria
Direcciones	Dirección de E/S — espacio separado del de memoria (típicamente más chico)	Dirección de memoria — mismo espacio que usan <code>MOV</code> y demás instrucciones normales
Datos	El dato leído/escrito del registro de E/S	Igual — el dato leído/escrito
Control	Señal que indica “acceso de E/S” (p.ej. <code>IO/M=1</code>) + <code>READ</code> o <code>WRITE</code>	Señal que indica “acceso de memoria” (<code>IO/M=0</code>) + <code>READ</code> o <code>WRITE</code> — no hay señal especial de E/S

En E/S aislada hay instrucciones dedicadas (`IN` / `OUT`) y un espacio de direcciones propio: una dirección de memoria `0x20` y una dirección de E/S `0x20` no colisionan, son espacios distintos. En E/S mapeada a memoria el controlador ocupa un rango del mismo espacio de direcciones

que la RAM y se accede con las instrucciones normales — no hace falta instrucción especial, pero se “gasta” espacio de direcciones de memoria. La diferencia en el bus de control **no es una señal adicional** en el caso mapeado a memoria, sino la **ausencia** de la señal distintiva de E/S: es el diseño físico del mapa de direcciones el que separa ambos mundos.

(b) Propósito del controlador de interrupciones.

Actúa como intermediario entre múltiples controladores de E/S y la única entrada `INT` de la CPU. Cuando algún dispositivo solicita interrupción, el controlador de interrupciones activa `INT` hacia la CPU y, cuando esta responde con `INTA`, es **el controlador de interrupciones** (no el dispositivo directamente) quien coloca en el bus de datos el identificador de la línea `IRQ` de origen. Así centraliza la identificación del origen y la gestión de prioridades entre múltiples fuentes — algo que la CPU, con una única entrada `INT`, no podría resolver por sí sola.

(c) Mecanismo INT/INTA y Daisy Chain. ¿Hay prioridad?

Con una única entrada `INT` (conectada en OR cableado a todos los controladores), la CPU no sabe por sí sola quién generó el pedido. Al aceptarlo, sube la señal `INTA` (*interrupt acknowledge*). En **Daisy Chain**, `INTA` se propaga en cadena, dispositivo por dispositivo, en un orden físico fijo: el primero de la cadena con un pedido pendiente “atrapa” la señal y no la deja pasar al siguiente, identificándose así ante la CPU (coloca su identificador en el bus de datos).

Sí existe prioridad, y es fija, determinada por la posición física en la cadena: el dispositivo más cercano al inicio (el que recibe primero `INTA`) tiene la mayor prioridad, porque siempre tiene la oportunidad de interceptar la señal antes que los siguientes — sin importar el orden en que hayan pedido la interrupción.

(d) Máquina dedicada vs. no dedicada. Consecuencias sobre el diseño.

- **Dedicada:** el procesador ejecuta un único sistema (todas las rutinas —principal e interrupciones— las escribe el mismo equipo). Se pueden fijar por convención reglas de uso de recursos compartidos (p.ej. “la rutina de interrupción nunca usa BX”) sin necesidad de proteger el contexto de “otros” programas, porque no los hay.
- **No dedicada:** coexisten programas de propósito distinto. Una rutina de interrupción no puede asumir nada del programa interrumpido: **debe guardar y restaurar explícitamente** (típicamente con `PUSH` / `POP`) cualquier registro que use.

Consecuencia práctica de diseño: en máquina dedicada, el polling puede hacerse directamente en el `while(1)` del programa principal sin culpa (no hay “otra cosa” que la CPU debería estar haciendo) — así se resuelven los Ejercicios 5 y 7 de este práctico, ambos explícitamente

dedicados. En máquina no dedicada ese mismo polling bloquearía injustamente a todo el resto del sistema, y ahí es donde conviene usar interrupciones en lugar de espera activa.

Ejercicio 1 — Polling del teclado

Registros: `ESTADO_TECLADO` y `DATO_TECLADO`, un byte cada uno, solo lectura. El hardware pone el bit 7 (MSB) de `ESTADO_TECLADO` en 1 cuando hay carácter disponible, y lo pone en 0 automáticamente al leerse `DATO_TECLADO`.

```
#define MAX_CHAR 1024

char buffer[MAX_CHAR];
int tope = 0;

void leerTeclado() {
    while (1) {
        // Mientras el bit 7 de ESTADO_TECLADO esté en 0, no hay dato: sigo esperando.
        while ((in(ESTADO_TECLADO) & 0x80) == 0)
            ; // espera activa (polling)

        // Bit 7 en 1: hay un caracter disponible.
        buffer[tope] = in(DATO_TECLADO); // leerlo apaga el bit 7 automáticamente (lo
        // hace el hardware)
        tope = (tope + 1) % MAX_CHAR;    // buffer circular
    }
}
```

Verificación de la condición (con `python -c "print(hex(0x80&0x80))"` → `0x80`, y `print(hex(0x00&0x80))` → `0x00`):

- `ESTADO_TECLADO = 0x80` (hay dato): `0x80 & 0x80 = 0x80`, que `== 0` es falso → sale del `while` interno, lee el dato. ✓
- `ESTADO_TECLADO = 0x00` (no hay dato): `0x00 & 0x80 = 0x00`, que `== 0` es verdadero → sigue esperando. ✓

Error real #1 en la solución oficial (Ejercicio 1): el PDF escribe `while`

`((in(ESTADO_TECLADO) & 0x80) == 1);`. Esto es incorrecto: `in(ESTADO_TECLADO) & 0x80` solo puede valer `0x00` o `0x80` (128 decimal) — **nunca** `1`. Comparar contra `1` hace que la condición sea **siempre falsa**: el bucle de espera nunca esperaría, y el programa leería

`DATO_TECLADO` incluso sin dato disponible (basura). Verificado: `python -c "print(hex(0x80&0x80))"` → `0x80`, no `1`. La corrección es comparar contra `0`, como se hizo arriba.

Ejercicio 2 — Bomba de agua (activar / desactivar)

Puerto BOMBA (lectura/escritura normal, 1 byte). Bit 2 = orden de encendido (CPU→bomba). Bit 0 = sensor externo, confirma si la bomba está realmente activa. Bit 4 = LED que refleja el estado real.

```
void activarBomba() {
    char bomb = in(BOMBA);
    bomb = bomb | 0x04;          // bit2=1: ordeno encender
    out(BOMBA, bomb);

    // Polling: espero a que el sensor (bit0) confirme que ya arrancó.
    while ((in(BOMBA) & 0x01) == 0)
        ;

    bomb = in(BOMBA);
    bomb = bomb | 0x10;          // bit4=1: prendo el LED de confirmación
    out(BOMBA, bomb);
}

void desactivarBomba() {
    char bomb = in(BOMBA);
    bomb = bomb & 0xFB;          // bit2=0: ordeno apagar (0xFB = ~0x04)
    out(BOMBA, bomb);

    // Polling: espero a que el sensor (bit0) confirme que ya se detuvo.
    while ((in(BOMBA) & 0x01) != 0)
        ;

    bomb = in(BOMBA);
    bomb = bomb & 0xEF;          // bit4=0: apago el LED (0xEF = ~0x10)
    out(BOMBA, bomb);
}
```

Verificación de máscaras (`python -c "print(hex((~0x04)&0xFF))"` → `0xfb`; `python -c "print(hex((~0x10)&0xFF))"` → `0xef`): `0xFB` = complemento de `0x04` (`0000 0100` invertido = `1111 1011`) ✓. `0xEF` = complemento de `0x10` (`0001 0000` invertido = `1110 1111`) ✓.

Error real #2 en la solución oficial (Ejercicio 2): el PDF escribe `while (in(BOMBA) & 0x01 != 1);` y `while (in(BOMBA) & 0x01 == 1);`, **sin paréntesis** alrededor del `&`. En C, `!=` y `==` tienen **mayor precedencia** que `&`. Entonces `in(BOMBA) & 0x01 != 1` se evalúa como `in(BOMBA) & (0x01 != 1)`. Como `0x01 != 1` es falso (`0x01` y `1` son el mismo valor), la expresión completa queda `in(BOMBA) & 0` = **siempre 0** (verificado: `python -c "print(int(0x01!=1))" → 0`, `python -c "print(0x05 & 0)" → 0`). El primer `while` (condición `!= 1`) nunca esperaría; el segundo, `in(BOMBA) & 0x01 == 1`, se evalúa como `in(BOMBA) & (0x01==1) = in(BOMBA) & 1` — en este caso particular coincide numéricamente con lo que se buscaba (porque la máscara de interés es justo el bit 0, y comparar contra 1 da el mismo AND), pero es una casualidad del bit elegido, no una expresión correcta: **siempre hay que poner los paréntesis explícitos**, como se hizo arriba.

Ejercicio 3 — Rutinas por secuencia de teclado (<ESC><n>)

Máquina de estados con tres estados: esperando <ESC>, esperando el número, y ejecutando una rutina utilitaria.

```

#define ESPERANDO_ESC 1
#define ESPERANDO_NUM 2
#define EJECUTANDO_RUTINA 3
#define ESC 0x1B // codigo del caracter especial <ESC> (valor de ejemplo, dato del
enunciado)

// Variables globales (persisten entre invocaciones de la interrupción):
int estado = ESPERANDO_ESC;
rutina* ret;

void interrupcionTeclado() {
    char tecla = in(TECLA);

    if (estado == ESPERANDO_ESC) {
        if (tecla == ESC)
            estado = ESPERANDO_NUM;
        // cualquier otra tecla mientras se espera <ESC>: se ignora, no cambia el estado
    }
    else if (estado == ESPERANDO_NUM) {
        if (tecla == '1') {
            ret = padr();
            estado = EJECUTANDO_RUTINA;
            jmp(radr("RUTINA1"));
        }
        else if (tecla == '2') {
            ret = padr();
            estado = EJECUTANDO_RUTINA;
            jmp(radr("RUTINA2"));
        }
        else if (tecla == '3') {
            ret = padr();
            estado = EJECUTANDO_RUTINA;
            jmp(radr("RUTINA3"));
        }
        else if (tecla == ESC) {
            // <ESC><ESC>: se queda esperando el numero (no rompe la secuencia)
            estado = ESPERANDO_NUM;
        }
        else {
            // secuencia invalida: reinicio
            estado = ESPERANDO_ESC;
        }
    }
    else { // estado == EJECUTANDO_RUTINA
        if (tecla == ESC) {
            estado = ESPERANDO_ESC;
            jmp(ret); // vuelve al programa X en el punto donde fue interrumpido
        }
        // cualquier otra tecla mientras se ejecuta la rutina: se ignora (la rutina la
        consume ella misma si le interesa)
    }
}

```

```
}
}
```

Error real #3, adicional, encontrado en la solución oficial (Ejercicio 3): las macros están escritas con un `;` **colgante**: `#define ESPERANDO_ESC 1;`. Esto es un bug real de preprocesador en C: el `;` forma parte del texto que reemplaza a `ESPERANDO_ESC` en cada uso. Una comparación como `if (estado == ESPERANDO_ESC)` se expandiría a `if (estado == 1;)` — **error de sintaxis** (un `;` no puede aparecer dentro de los paréntesis de un `if`). La corrección es simplemente no poner `;` al final del `#define`, como se hizo arriba.

Nota de diseño: el estado `EJECUTANDO Rutina` es innecesario para decidir *qué* rutina ejecutar (eso ya se resolvió al hacer `jmp`), pero es imprescindible para que la propia interrupción de teclado sepa que la tecla `<ESC>` que está por llegar debe interpretarse como “volver”, y no como el inicio de una nueva secuencia `<ESC><n>` — sin ese estado, la máquina no podría distinguir ambos significados de `<ESC>`.

Ejercicio 4 — Tres dispositivos, una sola línea de interrupción (atención por nivel)

A) PRIORIDAD FIJA: $PRI(DISPLAY1) > PRI(DISPLAY2) > PRI(DISPLAY3)$

```
void interrupciones() {
    disable();
    if ((in(ESTADO_1) & 0x01) != 0)
        atencionDispositivo1();
    else if ((in(ESTADO_2) & 0x01) != 0)
        atencionDispositivo2();
    else
        atencionDispositivo3();
    enable();
}
```

El orden de los `if` es la prioridad: si 1 y 2 piden a la vez, siempre se atiende primero al 1, porque se consulta primero. (Nota: como el atendimento es **por nivel**, si el dispositivo 3 pidió pero 1 o 2 también, esta rutina solo atiende a uno por invocación — el dispositivo no atendido

mantendrá su señal de pedido en 1, por ser atención por nivel, y volverá a generar una nueva interrupción en cuanto se rehabiliten las interrupciones, sin perderse.)

B) PRIORIDAD VARIABLE: EXISTE `PRI(DISPOSITIVO)`

En lugar de anidar manualmente 6 combinaciones de `if` (como hace la solución oficial, con varios tipos de variables — ver nota de abajo), conviene **ordenar los 3 dispositivos por prioridad una sola vez** y recorrerlos en ese orden hasta encontrar uno con pedido pendiente:

```
void interrupciones() {
    disable();

    int disp[3] = {1, 2, 3};
    int p[3];
    p[0] = pri(1);
    p[1] = pri(2);
    p[2] = pri(3);

    // Ordeno los 3 dispositivos por prioridad descendente (selection sort, 3 elementos:
    // O(1)).
    for (int i = 0; i < 3; i = i + 1) {
        int max = i;
        for (int j = i + 1; j < 3; j = j + 1)
            if (p[j] > p[max])
                max = j;
        int tmpD = disp[i]; disp[i] = disp[max]; disp[max] = tmpD;
        int tmpP = p[i];   p[i]   = p[max];   p[max]   = tmpP;
    }

    // Recorro en orden de prioridad y atiendo al primero con pedido pendiente.
    for (int i = 0; i < 3; i = i + 1) {
        if (disp[i] == 1 && (in(ESTADO_1) & 0x01) != 0) { atencionDispositivo1(); break; }
        if (disp[i] == 2 && (in(ESTADO_2) & 0x01) != 0) { atencionDispositivo2(); break; }
        if (disp[i] == 3 && (in(ESTADO_3) & 0x01) != 0) { atencionDispositivo3(); break; }
    }

    enable();
}
```

Discrepancia encontrada en la solución oficial (Ejercicio 4-B): el código del PDF tiene una variable mal escrita (`priotario` en vez de `prioritario`, y `priorario` / `prioriatorio` en otras ramas) — serían errores de compilación (identificador no declarado). Además, la

lógica está duplicada 3 veces con más de 40 líneas de `if` anidados para un problema que se resuelve con un simple ordenamiento de 3 elementos seguido de un recorrido — el enfoque de arriba es equivalente mucho más corto y sin ese riesgo de typos.

Sobre la precedencia en este ejercicio: todas las comparaciones de este ejercicio son

`(in(ESTADO_n) & 0x01) != 0` — la máscara de interés es el **bit 0**. Si se escribiera sin paréntesis, `in(ESTADO_n) & 0x01 != 0` se evaluaría como `in(ESTADO_n) & (0x01 != 0) = in(ESTADO_n) & 1`, que en este caso puntual **da el mismo resultado** que `(in(ESTADO_n)&0x01)!=0` porque la máscara es exactamente `0x01` (a diferencia del Ejercicio 1, donde la máscara era `0x80` y el resultado hubiera sido distinto). Aun así, **siempre** hay que escribir los paréntesis explícitos — apoyarse en esta coincidencia numérica es una mala práctica que deja de funcionar en cuanto la máscara deja de ser `0x01` (como pasó en el Ejercicio 1 real).

Ejercicio 5 — Concentrador de comunicaciones (4 entradas, 1 salida)

Direcciones dadas: `EST_CONT_n` / `DATO_n` ($n=0..3$) de solo lectura, `DATO_S` (`0x20`) de solo escritura. Bit 0 de `EST_CONT_n` indica dato disponible; se apaga solo al leer `DATO_n`. Procesador dedicado.

```

#define MAX_BUFFER 1024

char bufferLinea[MAX_BUFFER]; // guarda el numero de linea (n)
char bufferDato[MAX_BUFFER];  // guarda el dato de esa linea
int  frente = 0;               // proximo a sacar (FIFO)
int  fondo  = 0;               // proximo lugar libre
int  cantidad = 0;             // cantidad de pares (n, dato) pendientes en el buffer

int  salidaOcupada = 0;        // 0 = libre, 1 = transmitiendo

void main() {
    disable();
    // instalar intEntrada() e intSalida() en el vector de interrupciones (una por cada
    controlador)
    enable();
    while (1)
        ; // procesador dedicado: todo el trabajo real ocurre en las interrupciones
}

void intEntrada() {
    char direccionesEstado[4] = { EST_CONT_0, EST_CONT_1, EST_CONT_2, EST_CONT_3 };
    char direccionesDato[4]   = { DATO_0, DATO_1, DATO_2, DATO_3 };

    for (int n = 0; n < 4; n = n + 1) {
        if ((in(direccionesEstado[n]) & 0x01) != 0) {
            char dato = in(direccionesDato[n]); // leer DATO_n apaga el bit automatica-
mente

            if (salidaOcupada == 0 && cantidad == 0) {
                // Nada pendiente y la salida esta libre: transmito directo.
                salidaOcupada = 1;
                out(DATO_S, n);
                out(DATO_S, dato);
            }
            else if (cantidad < MAX_BUFFER) {
                // Encolo el par (n, dato) al fondo del buffer FIFO.
                bufferLinea[fondo] = n;
                bufferDato[fondo]  = dato;
                fondo = (fondo + 1) % MAX_BUFFER;
                cantidad = cantidad + 1;
            }
            // si el buffer esta lleno, se descarta (segun enunciado)
        }
    }
}

void intSalida() {
    // Termino de transmitir el ultimo byte que se pidio (n, o el dato -- ver nota de
    diseno).
    if (cantidad > 0) {

```

```

    salidaOcupada = 1;
    char n      = bufferLinea[frente];
    char dato = bufferDato[frente];
    frente = (frente + 1) % MAX_BUFFER;
    cantidad = cantidad - 1;
    out(DATO_S, n);
    out(DATO_S, dato);
} else {
    salidaOcupada = 0;
}
}

```

Discrepancias reales encontradas en la solución oficial (Ejercicio 5):

1. **Variable no inicializada usada como flag:** el arreglo local `char hayDato[4]` se declara sin inicializar dentro de `intEntrada()`; si un controlador no tiene dato disponible, la posición correspondiente de `hayDato[i]` **nunca se escribe** y queda con basura de la pila — el `if (hayDato[i])` posterior puede evaluar como verdadero por casualidad, procesando un dato que no llegó. Hay que inicializar explícitamente `hayDato[i] = 0` para los 4 índices antes de los `if`.
2. **Nombre de variable inconsistente:** dentro del `for` que llena el buffer se escribe `buffer[largoBuffer+1] = ...`, pero la variable global declarada es `bufferSalida` — `buffer` no existe, error de compilación.
3. **Orden LIFO en vez de FIFO — bug de diseño real:** `intSalida()` toma el **último** elemento agregado (`posInfo = largoBuffer; largoBuffer -= 1;`), es decir, atiende el buffer como una **pila** (LIFO). Como los datos se encolan en pares `(n, dato)` en orden de llegada, sacarlos en orden inverso transmite primero el `dato` más reciente y en general rompe por completo el orden requerido “primero n, luego el dato” para cada línea, y el orden entre líneas. Además cada llamada a `intSalida()` transmite un solo byte del par, no el par completo. La versión de arriba corrige esto usando una cola circular (FIFO) que siempre transmite el par `(n, dato)` completo y en el orden en que llegó.

Ejercicio 6 — Iluminación de pasillos (botón + timer)

`boton()` se invoca por interrupción al presionar cualquier botón; `tiempo()` por un timer de 1 Hz. El bit 0 de `0x20` enciende/apaga todas las luces.

```

#define APAGADA 0
#define ENCENDIDA 1

char estado = APAGADA;
int contador; // segundos restantes hasta apagar

void boton() {
    disable();
    if (estado == APAGADA) {
        estado = ENCENDIDA;
        char byte = in(0x20);
        byte = byte | 0x01; // enciendo el bit 0
        out(0x20, byte);
    }
    contador = 45; // (re)inicio la cuenta regresiva, este prendida o no
    enable();
}

void tiempo() {
    disable();
    if (estado == ENCENDIDA) {
        contador = contador - 1;
        if (contador == 0) {
            estado = APAGADA;
            char byte = in(0x20);
            byte = byte & 0xFE; // apago el bit 0, sin tocar los demas bits (0xFE =
~0x01)
            out(0x20, byte);
        }
    }
    enable();
}

```

Verificación de la máscara: `python -c "print(hex((~0x01)&0xFF))"` → `0xfe` ✓.

Error real #4, adicional, encontrado en la solución oficial (Ejercicio 6): la primera versión del PDF escribe `if (estado = ENCENDIDA)` y `if (seg = 0)` — con **un solo =** (asignación) en vez de `==` (comparación). En C esto es una **asignación** que además se evalúa como verdadera si el valor asignado es distinto de cero: `if (estado = ENCENDIDA)` (con `ENCENDIDA=1`) siempre le asigna `1` a `estado` y siempre entra al `if` — la condición nunca depende de si las luces estaban realmente encendidas. Peor aún, `if (seg = 0)` le asigna `0` a `seg` y la condición se evalúa como **falsa** (0), así que ese `if` interno nunca se cumple: **las luces jamás se apagarían automáticamente**, contradiciendo directa-

mente el objetivo del ejercicio. (El propio PDF trae, más abajo en la misma página, una segunda versión ya corregida con `==` y un contador `timeout` — la resolución de arriba sigue ese mismo esquema correcto, con nombres more explícitos.)

Truco de verificación rápida para este tipo de bug: en cualquier `if (x = y)`, si `y` es una constante no nula, la condición **siempre** es verdadera (asignación + valor no-cero); si `y` es `0`, la condición **siempre** es falsa. Ninguno de los dos casos depende del valor previo de `x` — esa es la señal inequívoca de que falta un `=`.

Ejercicio 7 — Sistema de seguridad de gas (sensor + extractor + válvula)

Procesador **dedicado**. `SENSOR` (solo lectura): bit 0 = concentración peligrosa. `ACCESORIO` (lectura/escritura): bit 0 = extractor, bit 1 = válvula (se cierra escribiendo 0, y una vez cerrada solo se abre manualmente — el software nunca vuelve a poner ese bit en 1). Timer a 10 Hz invoca `tiempo()`.

```

#define SIN_FUGA 0
#define CON_FUGA 1
#define CERRADA 2
#define REINICIAR 3 // fuga finalizada con la valvula ya cerrada: listo para reabrir
manualmente

#define TICKS_30SEG 300 // 30 seg * 10 Hz = 300 interrupciones del timer

char estado = SIN_FUGA;
int contador; // ticks de timer desde que comenzo la fuga actual

void main() {
    disable();
    // instalar tiempo() como rutina de interrupcion del timer (10 Hz)
    estado = SIN_FUGA;
    enable();
    while (1)
        ; // procesador dedicado
}

void tiempo() {
    disable();
    char sens = in(SENSOR);
    char acc = in(ACCESORIO);
    char huboFuga = (sens & 0x01) != 0;

    if (estado == SIN_FUGA) {
        if (huboFuga) {
            estado = CON_FUGA;
            contador = 0;
            acc = acc | 0x01; // prendo el extractor
            out(ACCESORIO, acc);
        }
    }
    else if (estado == CON_FUGA) {
        contador = contador + 1;
        if (!huboFuga) {
            estado = SIN_FUGA;
            acc = acc & 0xFE; // apago el extractor (0xFE = ~0x01)
            out(ACCESORIO, acc);
        }
        else if (contador > TICKS_30SEG) {
            estado = CERRADA;
            acc = acc & 0xFD; // cierro la valvula (0xFD = ~0x02)
            out(ACCESORIO, acc);
        }
    }
    else if (estado == CERRADA) {
        if (!huboFuga) {
            estado = REINICIAR;
        }
    }
}

```

```

        acc = acc & 0xFE;          // apago el extractor, la valvula sigue cerrada
        out(ACCESORIO, acc);
    }
}
// estado == REINICIAR: la valvula quedo cerrada manualmente hasta que un operario la
reabra;
// no hay mas transiciones automaticas posibles desde aqui.

enable();
}

```

Verificación de máscaras: `python -c "print(hex((~0x01)&0xFF))"` → `0xfe`; `python -c "print(hex((~0x02)&0xFF))"` → `0xfd`; `30*10=300` ticks ✓.

Discrepancias reales encontradas en la solución oficial (Ejercicio 7): el código asigna `estado = ABIERTO_SIN_FUGA;` y `estado = CERRADO;`, pero las constantes definidas al inicio son `ABIERTA_SIN_FUGA` y `CERRADA` (género distinto) — son identificadores **no declarados**, error de compilación por typo. Además, igual que en los Ejercicios 4 y 5, hay comparaciones sin paréntesis (`sens & 0x01 == 1`, `sens & 0x01 == 0`) que, por el mismo motivo que en el Ejercicio 4, dan casualmente el resultado numérico correcto porque la máscara de interés es `0x01` — pero no deben tomarse como plantilla: los paréntesis explícitos de la versión de arriba son la forma correcta y general. El PDF además incluye una segunda versión alternativa (con nombres `TRUE / FALSE`, `ESCAPE_GAS`) en letra muy pequeña dentro de la misma página, que no se pudo verificar carácter por carácter con certeza — por eso se optó por reconstruir una versión propia, completamente verificada, en lugar de transcribirla.

Nota de diseño: a diferencia de la solución oficial (que separaba el polling del sensor en un `while` del programa principal, llamando a una función `sistemaGas()` en cada vuelta), acá se aprovecha que el enunciado ya da un timer a 10 Hz — el polling del sensor se hace directamente **dentro de** `tiempo()`, que de todas formas se ejecuta 10 veces por segundo; esto evita necesitar dos mecanismos de repetición (un `while(1)` de polling puro y un timer) cuando uno solo alcanza y ya está disponible. Es una decisión de diseño válida y más simple, no la única posible.

Ejercicio 8 — Caja de seguridad con combinación

⚠ Nota de transparencia: este ejercicio **no tiene solución en el PDF oficial** de este práctico (el material termina en el Ejercicio 5/7); la resolución que sigue es propia, construida y verificada desde cero a partir del enunciado.

Registros: `TECLADO` (solo lectura, 1 byte): dígitos codificados en los 4 bits más significativos, tecla `ENTER` = `0xD0` (código completo, no solo el nibble alto). `BARRAS` (solo escritura, 1 byte): escribir 1 en el bit más significativo extiende (cierra) las barras; escribir 1 en el bit menos significativo las retrae (abre). `DISPLAY` (solo escritura, 4 bytes): BCD empaquetado (2 dígitos por byte), guión de error = `0xFF` por byte (4 bytes en `0xFF` = 8 guiones), blanco = `0xEE` por byte. Timer a 2 Hz invoca `tiempo()`.

```

#define MAX_DIGITOS 8
#define ENTER_CODE 0xD0

#define CAJA_ABIERTA 0
#define CAJA_CERRADA 1
#define CAJA_BLOQUEADA 2

#define TICKS_INACTIVIDAD 10    // 5 seg * 2 Hz = 10 ticks sin tecla -> vaciar
#define TICKS_BLOQUEO 14400    // 2 horas * 3600 seg/h * 2 ticks/seg = 14400

char estado = CAJA_ABIERTA;

char clave[MAX_DIGITOS];    // combinacion vigente para abrir
int largoClave = 0;

char buffer[MAX_DIGITOS];    // digitos ingresados en el intento actual
int largoBuffer = 0;

int fallosConsecutivos = 0;
int contadorInactividad = 0;
int contadorBloqueo = 0;

void main() {
    disable();
    // instalar tecla() y tiempo() en el vector de interrupciones
    estado = CAJA_ABIERTA;
    largoBuffer = 0;
    largoClave = 0;
    fallosConsecutivos = 0;
    contadorInactividad = 0;
    apagarDisplay();
    enable();

    while (1)
        ; // dedicado: todo el trabajo ocurre en tecla() y tiempo()
}

void tecla() {
    disable();
    char byte = in(TECLADO);
    contadorInactividad = 0;    // cualquier tecla reinicia el timeout de inactividad

    if (estado != CAJA_BLOQUEADA) {
        if (byte == ENTER_CODE)
            procesarEnter();
        else {
            char digito = (byte >> 4) & 0x0F;    // 4 bits mas significativos
            agregarDigito(digito);
        }
    }
}

```

```

    // si esta bloqueada, cualquier tecla se ignora
    enable();
}

void agregarDigito(char d) {
    if (largoBuffer < MAX_DIGITOS) {
        buffer[largoBuffer] = d;
        largoBuffer = largoBuffer + 1;
    } else {
        // ya hay 8 digitos: descarto el mas antiguo (shift) y agrego el nuevo al final
        for (int i = 0; i < MAX_DIGITOS - 1; i = i + 1)
            buffer[i] = buffer[i + 1];
        buffer[MAX_DIGITOS - 1] = d;
    }
}

int coincideConClave() {
    if (largoBuffer != largoClave)
        return 0;
    for (int i = 0; i < largoClave; i = i + 1)
        if (buffer[i] != clave[i])
            return 0;
    return 1;
}

void procesarEnter() {
    if (estado == CAJA_ABIERTA) {
        if (largoBuffer < 4) {
            // codigo invalido (menos de 4 digitos): no hace nada, segun enunciado
            return;
        }
        for (int i = 0; i < largoBuffer; i = i + 1)
            clave[i] = buffer[i];
        largoClave = largoBuffer;

        out(BARRAS, 0x80);          // barras extendidas (MSB=1): cierro
        estado = CAJA_CERRADA;
        largoBuffer = 0;
    }
    else { // estado == CAJA_CERRADA
        if (coincideConClave()) {
            out(BARRAS, 0x01);      // barras retraidas (LSB=1): abro
            estado = CAJA_ABIERTA;
            fallosConsecutivos = 0;
        } else {
            fallosConsecutivos = fallosConsecutivos + 1;
            mostrarError();
            if (fallosConsecutivos == 3) {
                estado = CAJA_BLOQUEADA;
                contadorBloqueo = 0;
            }
        }
    }
}

```

```

    }
    largoBuffer = 0;
}

void mostrarError() {
    out(DISPLAY, 0xFF);
    out(DISPLAY + 1, 0xFF);
    out(DISPLAY + 2, 0xFF);
    out(DISPLAY + 3, 0xFF);
}

void apagarDisplay() {
    out(DISPLAY, 0xEE);
    out(DISPLAY + 1, 0xEE);
    out(DISPLAY + 2, 0xEE);
    out(DISPLAY + 3, 0xEE);
}

void tiempo() {
    disable();
    if (estado == CAJA_BLOQUEADA) {
        contadorBloqueo = contadorBloqueo + 1;
        if (contadorBloqueo >= TICKS_BLOQUEO) {
            estado = CAJA_CERRADA; // se levanta el bloqueo; la caja sigue cerrada
            fallosConsecutivos = 0;
        }
    }
    else {
        contadorInactividad = contadorInactividad + 1;
        if (contadorInactividad >= TICKS_INACTIVIDAD) {
            largoBuffer = 0;
            apagarDisplay();
            contadorInactividad = 0;
        }
    }
    enable();
}

```

Verificaciones numéricas (python): $5*2=10$ ticks de inactividad a 2 Hz ✓; $2*3600*2=14400$ ticks para 2 horas de bloqueo a 2 Hz ✓; para `byte=0x30` (dígito 3 en el nibble alto), $(0x30 >> 4) \& 0xF = 0x3$ ✓; `ENTER_CODE=0xD0` se compara por **igualdad exacta contra el byte completo**, nunca extrayendo su nibble alto como si fuera un dígito ($(0xD0 >> 4) \& 0xF = 0xD$, que no es un dígito 0–9 válido — por eso el chequeo de `ENTER` va **antes** y aparte de la extracción de dígito).

Notas de diseño explícitas (decisiones no fijadas unívocamente por el enunciado):

1. El enunciado no exige mostrar en el display los dígitos a medida que se tipean — solo especifica el comportamiento de error (guiones) y de vaciado tras 5 s de inactividad. Se optó por **no** ecoar dígitos en pantalla durante la carga (diseño típico de teclados de seguridad, que no muestran el código tipeado), documentando la decisión acá en vez de inventar semántica no pedida.
2. "Últimos 8 dígitos" se implementa como una ventana deslizante de tamaño 8 (descarta el más viejo al llegar el 9°), consistente con la redacción "solo se considerarán los últimos 8 ingresados antes de presionar ENTER".
3. El desbloqueo tras las 2 horas deja la caja en estado `CAJA_CERRADA` (no la abre automáticamente) — el enunciado solo dice que "quedará bloqueada por 2 horas", no que se abra sola después; abrir requiere volver a ingresar la combinación correcta.

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
Teoría	Buses (direcciones/datos/control), INT/INTA + Daisy Chain, máquina dedicada vs. no dedicada
1, 2	Polling con máscara AND + comparación — errores de precedencia y de valor de comparación corregidos
3	Máquina de estados por secuencia de teclas, vía interrupción de teclado
4	Identificación por software (varios dispositivos, una línea) + prioridad fija y variable
5	Concentrador con buffer — cola FIFO circular (corrigiendo un LIFO incorrecto de la solución oficial)
6, 7	Interrupciones de timer + evento externo combinadas, contador de tiempo con máscaras AND/OR
8	Máquina de estados completa (abierta/cerrada/bloqueada) con timeouts de inactividad y de bloqueo, sin material oficial de referencia

Para el examen: el patrón más transferible de este práctico es "**leer completo** → **aplicar máscara con paréntesis explícitos** → **escribir completo**" para cualquier registro de E/S con bits mezclados, y la regla de tolerancia cero de precedencia en C (`&` / `|` / `^`) se evalúan **después** de `==` / `!=`, así que toda comparación de bits necesita paréntesis alrededor del AND/OR/XOR). El segundo patrón transferible es que casi todo diseño de este tipo se reduce a

una **máquina de estados** con transiciones disparadas por interrupciones (evento externo + timer), donde el punto crítico a verificar siempre es: ¿quién reinicia el contador de tiempo?, ¿qué pasa si dos rutinas leen/escriben el mismo registro sin encontrarse en un estado intermedio raro?, y — como se vio en varios de los “errores adicionales” de este práctico — revisar que cada macro, cada nombre de constante y cada `==` / `=` esté bien escrito, porque un typo de compilación es tan grave en un parcial como un error conceptual.